M ABUBAKER BARKET

Machine Learning Lab 01 (241410)

February 23, 2026

Task 1:

```python
import pandas as pd
df = pd.read_csv('/content/housing_price_dataset.csv')
print("Dataset loaded successfully. Displaying the first 5 rows:")
df.head()
```

```
Dataset loaded successfully. Displaying the first 5 rows:
```

	SquareFeet	Bedrooms	Bathrooms	Neighborhood	YearBuilt	Price
0	2126	4	1	Rural	1969	215355.283618
1	2459	3	2	Rural	1980	195014.221626
2	1860	2	1	Suburb	1970	306891.012076
3	2294	2	1	Urban	1996	206786.787153
4	2130	5	2	Suburb	2001	272436.239065

Next steps: Generate code with df | New interactive sheet

Task 2:

[9]

```python
print("First 5 rows of the DataFrame:")
print(df.head())
print("\nShape of the DataFrame (rows, columns):")
print(df.shape)
print("\nData types of each column:")
print(df.dtypes)
```

```
First 5 rows of the DataFrame:
   SquareFeet  Bedrooms  Bathrooms Neighborhood  YearBuilt          Price
0        2126         4          1        Rural       1969  215355.283618
1        2459         3          2        Rural       1980  195014.221626
2        1860         2          1       Suburb       1970  306891.012076
3        2294         2          1        Urban       1996  206786.787153
4        2130         5          2       Suburb       2001  272436.239065

Shape of the DataFrame (rows, columns):
(50000, 6)

Data types of each column:
SquareFeet        int64
Bedrooms          int64
Bathrooms         int64
Neighborhood     object
YearBuilt         int64
Price           float64
dtype: object
```

Task 3:

```
[10]
0s
print("Missing values before handling:")
print(df.isnull().sum())

Missing values before handling:
SquareFeet      0
Bedrooms        0
Bathrooms       0
Neighborhood    0
YearBuilt       0
Price           0
dtype: int64
```

Task 4:

```
[11]
0s
df['house_age'] = 2025 - df['YearBuilt']
print("DataFrame with new 'house_age' feature:")
df.head()

DataFrame with new 'house_age' feature:
```

	SquareFeet	Bedrooms	Bathrooms	Neighborhood	YearBuilt	Price	house_age
0	2126	4	1	Rural	1969	215355.283618	56
1	2459	3	2	Rural	1980	195014.221626	45
2	1860	2	1	Suburb	1970	306891.012076	55
3	2294	2	1	Urban	1996	206786.787153	29
4	2130	5	2	Suburb	2001	272436.239065	24

Task 5:

Task 6:

[12]
0s

```python
unique_neighborhoods = df['Neighborhood'].unique()
neighborhood_mapping = {neighborhood: i for i, neighborhood in enumerate(unique_neighborhoods)}
df['Neighborhood_encoded'] = df['Neighborhood'].map(neighborhood_mapping)

print("DataFrame with new 'Neighborhood_encoded' feature:")
print(df.head())
```

```
DataFrame with new 'Neighborhood_encoded' feature:
   SquareFeet  Bedrooms  Bathrooms Neighborhood  YearBuilt          Price  \
0        2126         4          1        Rural       1969  215355.283618
1        2459         3          2        Rural       1980  195014.221626
2        1860         2          1       Suburb       1970  306891.012076
3        2294         2          1        Urban       1996  206786.787153
4        2130         5          2       Suburb       2001  272436.239065

   house_age  Neighborhood_encoded
0         56                     0
1         45                     0
2         55                     1
3         29                     2
4         24                     1
```

Task 7:

[13]
0s

```python
df['price_per_sqft'] = df['Price'] / df['SquareFeet']
print("DataFrame with new 'price_per_sqft' feature:")
df.head()
```

```
DataFrame with new 'price_per_sqft' feature:
```

	SquareFeet	Bedrooms	Bathrooms	Neighborhood	YearBuilt	Price	house_age	Neighborhood_encoded	price_per_sqft
0	2126	4	1	Rural	1969	215355.283618	56	0	101.295994
1	2459	3	2	Rural	1980	195014.221626	45	0	79.306312
2	1860	2	1	Suburb	1970	306891.012076	55	1	164.995168
3	2294	2	1	Urban	1996	206786.787153	29	2	90.142453
4	2130	5	2	Suburb	2001	272436.239065	24	1	127.904338

Next steps: Generate code with df New interactive sheet

Task 8:

```
[14]
✓ 0s
import numpy as np

X = df[['SquareFeet', 'Bedrooms', 'Bathrooms', 'house_age', 'Neighborhood_encoded']].to_numpy()
y = df['Price'].to_numpy().reshape(-1, 1)

print("Shape of feature matrix X:", X.shape)
print("Shape of target vector y:", y.shape)
print("First 5 rows of X:\n", X[:5])
print("First 5 rows of y:\n", y[:5])
```

```
Shape of feature matrix X: (50000, 5)
Shape of target vector y: (50000, 1)
First 5 rows of X:
 [[2126    4    1   56    0]
 [2459    3    2   45    0]
 [1860    2    1   55    1]
 [2294    2    1   29    2]
 [2130    5    2   24    1]]
First 5 rows of y:
 [[215355.2836182 ]
 [195014.22162585]
 [306891.01207633]
 [206786.78715333]
 [272436.23906506]]
```

Task 9:

```
[15]
✓ 0s
from sklearn.preprocessing import MinMaxScaler

scaler = MinMaxScaler()

numerical_features = ['SquareFeet', 'Bedrooms', 'Bathrooms', 'house_age']

df[numerical_features] = scaler.fit_transform(df[numerical_features])

print("DataFrame with normalized numerical features (first 5 rows):")
print(df.head())
```

```
DataFrame with normalized numerical features (first 5 rows):
   SquareFeet  Bedrooms  Bathrooms Neighborhood  YearBuilt          Price  \
0    0.563282  0.666667        0.0        Rural       1969  215355.283618
1    0.729865  0.333333        0.5        Rural       1980  195014.221626
2    0.430215  0.000000        0.0       Suburb       1970  306891.012076
3    0.647324  0.000000        0.0        Urban       1996  206786.787153
4    0.565283  1.000000        0.5       Suburb       2001  272436.239065

   house_age  Neighborhood_encoded  price_per_sqft
0   0.732394                     0      101.295994
1   0.577465                     0       79.306312
2   0.718310                     1      164.995168
3   0.352113                     2       90.142453
4   0.281690                     1      127.904338
```

Task 10:

[16]
0s

```python
import numpy as np

feature_columns = ['SquareFeet', 'Bedrooms', 'Bathrooms', 'house_age', 'Neighborhood_encoded']
y = df['Price']

print("Pearson Correlation Coefficients with 'Price':")

for feature in feature_columns:
    x = df[feature]
    mean_x = np.mean(x)
    mean_y = np.mean(y)
    numerator = np.sum((x - mean_x) * (y - mean_y))
    denominator_x = np.sqrt(np.sum((x - mean_x)**2))
    denominator_y = np.sqrt(np.sum((y - mean_y)**2))
    denominator = denominator_x * denominator_y
    if denominator != 0:
        correlation = numerator / denominator
        print(f"  {feature}: {correlation:.4f}")
    else:
        print(f"  {feature}: Cannot compute correlation (denominator is zero)")
```

```
Pearson Correlation Coefficients with 'Price':
  SquareFeet: 0.7507
  Bedrooms: 0.0726
  Bathrooms: 0.0284
  house_age: 0.0023
  Neighborhood_encoded: 0.0164
```

<----------->